Python Syntax Reference

Day 1 — complete refresher

Prep materials — Anthropic Fellows OA

August 2026

Contents

Python Syntax Reference — Complete Refresher

A full sweep of Python syntax for an experienced engineer who's rusty. Everything is runnable — keep a REPL open. Sections marked **[OA]** are the ones that matter most for a CodeSignal-style systems test; sections marked **[skim]** are completeness, not priority.

Reading order if short on time: 2, 3, 4, 6, 7, 8, 11, 13 → then the rest.

---

1. Program structure & syntax basics

```
# Comments start with #
"""Module docstring — first statement in a file."""

x = 5                      # no declarations, no semicolons, no braces
```

```python
if x > 3:                   # colon opens a block
    print("big")            # INDENTATION defines the block (4 spaces)
    print("still inside")
print("outside")

# Line continuation
total = (1 + 2 +            # inside brackets: implicit, preferred
         3 + 4)
total = 1 + 2 + \
        3                   # backslash: legal, avoid

a = b = 0                   # chained assignment
a, b = 1, 2                 # tuple assignment
a, b = b, a                 # swap, no temp
x += 1; x -= 1; x *= 2; x //= 2; x **= 2; x %= 3   # no ++ or --

if (n := len(xs)) > 3:      # walrus := assigns inside an expression (3.8+)
    print(n)
```

Naming conventions (enforced by convention, not the compiler): `snake_case` for functions/variables, `PascalCase` for classes, `UPPER_SNAKE` for constants, `_private` by convention, `__mangled` in classes.

Scope: `global x` to rebind a module-level name inside a function; `nonlocal x` to rebind an enclosing function's local. Reading needs neither.

```python
counter = 0
def bump():
    global counter
    counter += 1
```

---

2. Numbers, operators, and arithmetic [OA]

```python
i = 42                 # int — arbitrary precision, never overflows
f = 3.14               # float — IEEE 754 double
c = 2 + 3j             # complex  [skim]
b = True               # bool IS an int subclass: True + True == 2

7 / 2      # 3.5    — true division, ALWAYS float
7 // 2     # 3      — floor division
-7 // 2    # -4     — floors toward NEGATIVE INFINITY (not toward zero!)
7 % 3      # 1
-7 % 3     # 2      — result takes the sign of the DIVISOR
2 ** 10    # 1024
divmod(7, 2)           # (3, 1) — quotient and remainder together

abs(-3); round(2.675, 2); round(0.5)   # round() is banker's rounding: 0
int(3.9); int(-3.9)    # 3, -3 — truncates toward zero (unlike //)
float("1.5"); int("42"); int("ff", 16)
```

Ceiling division — memorize both forms:

```python
import math
math.ceil(a / b)       # float path; can be wrong for huge ints
-(-a // b)             # exact integer path — prefer this
```

Float comparison is unreliable: `0.1 + 0.2 == 0.3` is `False`. Use `math.isclose(x, y)`. For money/exactness use `decimal.Decimal` or `fractions.Fraction`. **[skim]**

```
math.inf; -math.inf; math.nan          # nan != nan, always
math.floor(x); math.ceil(x); math.sqrt(x); math.log(x, base)
math.gcd(a, b); math.factorial(n); math.comb(n, k)
```

Bitwise: `& | ^ ~ << >>`. Chained comparison works as in math: `0 <= x < 10` is one expression, evaluating `x` once.

---

3. Strings [OA]

```
s = 'single'; s = "double"          # identical
s = """multi
line"""
s = r"raw\nno escape"               # raw — no backslash processing
b = b"bytes"                        # bytes, not str

# f-strings (3.6+) — your primary formatting and debugging tool
name, val = "t", 3.14159
f"{name}={val}"                     # 't=3.14159'
f"{val:.2f}"                        # '3.14'
f"{val:8.2f}"                       # '    3.14'  width 8, right-aligned
f"{name:<10}|{name:>10}|{name:^10}" # left / right / center pad
f"{42:05d}"                         # '00042'
f"{255:x} {255:b} {255:o}"          # hex, binary, octal
f"{0.256:.1%}"                      # '25.6%'
f"{val=}"                           # 'val=3.14159'  — debug form (3.8+)
f"{ {'a':1}['a'] }"                 # expressions allowed inside
```

Strings are **immutable** — every operation returns a new string.

```
s = "hello world"
len(s); s[0]; s[-1]; s[0:5]; s[::-1]; s[::2]
s.upper(); s.lower(); s.title(); s.capitalize(); s.swapcase()
s.strip(); s.lstrip(); s.rstrip(); s.strip(".,")
s.split(); s.split(","); s.split(",", 1); s.rsplit(); s.splitlines()
",".join(["a", "b"])                # 'a,b'  — join is a STRING method
s.replace("l", "L"); s.replace("l", "L", 1)
s.find("wor")      # 6, or -1 if absent
s.index("wor")     # 6, or raises ValueError
s.count("l"); "wor" in s
s.startswith("he"); s.endswith(("d", "x"))   # tuple of options allowed
s.isdigit(); s.isalpha(); s.isalnum(); s.isspace(); s.islower()
s.zfill(5); s.center(20, "-"); s.ljust(10); s.rjust(10)
s.removeprefix("hello "); s.removesuffix(" world")     # 3.9+
ord("a"); chr(97)                   # 97, 'a'
"".join(sorted(s))                  # canonical anagram key
```

GOTCHA: building a string in a loop with += is $O(n^2)$. Accumulate into a list and `"".join(parts)` at the end.

---

4. Lists [OA]

```
xs = [1, 2, 3]
xs = list(range(5))
```

```
xs = [0] * 5                         # [0,0,0,0,0]
grid = [[0] * 3 for _ in range(3)]  # 3x3 — CORRECT
grid = [[0] * 3] * 3                 # WRONG: three refs to the SAME row
```

```
xs.append(4)          # add one at end                O(1)
xs.extend([5, 6])     # add many  (xs += [5,6])       O(k)
xs.insert(0, 9)       # insert at index               O(n)
xs.pop()              # remove & return last          O(1)
xs.pop(0)             # remove & return first         O(n) -> use deque
xs.remove(9)          # delete FIRST match by value   O(n), ValueError if absent
del xs[0]; del xs[1:3]
xs.clear()
xs.index(3); xs.index(3, start, end)
xs.count(3)
xs.reverse()          # in place, returns None
xs.sort()             # in place, returns None
sorted(xs)            # new list
xs.copy(); xs[:]      # shallow copy
```

Slicing never raises on out-of-range bounds:

```
xs[1:3]; xs[:2]; xs[2:]; xs[-2:]; xs[::2]; xs[::-1]
xs[1:3] = [9, 9, 9]      # slice assignment can change length
xs[:] = [1, 2]           # replace contents IN PLACE (keeps aliases valid)
```

```
a + b                 # concatenation (new list)
a * 3
3 in a                # O(n) membership — use a set for hot loops
len(a); min(a); max(a); sum(a)
any(x > 2 for x in a); all(x > 0 for x in a)
list(zip(a, b)); list(enumerate(a)); list(reversed(a))
```

---

5. Tuples, sets, frozensets

```
t = (1, 2); t = 1, 2                # parens optional
single = (1,)                       # trailing comma REQUIRED
empty = ()
t[0]; len(t); t + (3,); t * 2       # immutable: no append/assign
a, b = t                            # unpacking
first, *rest = (1, 2, 3)            # rest == [2, 3]
a, (b, c) = 1, (2, 3)               # nested unpacking
```

Tuples are hashable (if their contents are), so they work as dict keys and set members, and they compare **element by element** — the basis of every multi-key sort. **[OA]**

```
(1, 5) < (1, 9) < (2, 0)            # True
```

Named tuples for readable records: **[skim]**

```
from collections import namedtuple
Point = namedtuple("Point", "x y")
p = Point(1, 2); p.x; p[0]
from typing import NamedTuple
class Point(NamedTuple):
    x: int
    y: int = 0
```

Sets — unordered, unique, O(1) membership:

```
s = {1, 2, 3}; s = set()            # {} is an empty DICT, not a set
s.add(4); s.discard(9)              # discard: no error if absent
s.remove(9)                         # KeyError if absent
s.pop()                             # arbitrary element
a | b   # union          a.union(b)
a & b   # intersection     a.intersection(b)
a - b   # difference
a ^ b   # symmetric difference
a <= b  # subset           a.issubset(b)
frozenset([1, 2])                   # immutable, hashable set
```

Only hashable things go in a set — no lists, and no plain dataclass instances (see the OOP notes).

---

6. Dicts [OA]

```
d = {"a": 1, "b": 2}
d = dict(a=1, b=2)
d = dict([("a", 1)])
d = {k: v for k, v in pairs}
d = dict.fromkeys(["a", "b"], 0)
```

```
d["a"]                  # KeyError if missing
d.get("z")              # None
d.get("z", 0)           # default
d.setdefault("k", []).append(1)     # get-or-create then use
d["c"] = 3; del d["c"]
d.pop("a"); d.pop("a", None)
d.popitem()             # removes & returns the LAST inserted pair
d.update({"c": 3}); d |= {"c": 3}   # merge (3.9+);  e = d | other
"a" in d                # checks KEYS, O(1)
len(d); d.clear(); d.copy()

d.keys(); d.values(); d.items()     # dynamic views, not lists
for k in d: ...                     # iterates KEYS
for k, v in d.items(): ...
sorted(d)                           # sorted keys
sorted(d.items(), key=lambda kv: -kv[1])    # sort by value desc
max(d, key=d.get)                   # key with the largest value
```

Insertion order is guaranteed (3.7+). Keys must be hashable.

GOTCHA: never add or delete keys while iterating a dict — iterate over `list(d)` or `list(d.items())` if you must mutate.

---

7. Control flow [OA]

```
if cond:
    ...
elif other:
    ...
else:
    ...
```

```
value = a if cond else b            # ternary

for x in xs: ...
for i in range(n): ...              # range(start, stop, step); stop exclusive
for i, x in enumerate(xs, start=1): ...
for a, b in zip(xs, ys): ...        # stops at the shorter one
for a, b in zip(xs, ys, strict=True): ...   # 3.10+: error on length mismatch
for k, v in d.items(): ...
for _ in range(3): ...              # _ = "I don't need this"

while cond: ...
while True:
    if done: break
    if skip: continue

for x in xs:
    if found: break
else:
    print("loop finished without break")     # for/else — rare but real

pass          # do-nothing placeholder
...           # Ellipsis, also usable as a placeholder
```

Boolean logic uses words and short-circuits:

```
a and b; a or b; not a
x = val or "default"        # falls back when val is falsy (careful with 0!)
0 <= x < 10                 # chained comparison
a is b; a is not b          # identity, not equality
x in xs; x not in xs
```

match (3.10+) — structural pattern matching: **[skim]**

```
match command.split():
    case ["go", direction]:
        move(direction)
    case ["quit"] | ["exit"]:
        stop()
    case {"type": "req", "id": rid}:      # dict pattern
        handle(rid)
    case Point(x=0, y=y):                 # class pattern
        ...
    case _:
        unknown()
```

---

8. Comprehensions & generators [OA]

```
[x * 2 for x in xs]                               # list
[x for x in xs if x > 0]                          # filter
[x if x > 0 else 0 for x in xs]                   # ternary BEFORE the `for`
[(i, j) for i in range(3) for j in range(3)] # nested loops, outer first
[[c for c in row] for row in grid]                # nested comprehension
{k: v for k, v in pairs}                          # dict
{v: k for k, v in d.items()}                      # invert a dict
{x % 3 for x in xs}                               # set
(x * 2 for x in xs)                               # GENERATOR — lazy, one pass
sum(r.tokens for r in reqs)                       # bare genexp as sole argument
```

Generators are lazy and memory-cheap; they can only be consumed once.

```
def countdown(n):
    while n > 0:
        yield n            # produces a value, suspends here
        n -= 1
    return                 # StopIteration

g = countdown(3)
next(g); next(g)
list(countdown(3))         # [3, 2, 1]

def flatten(nested):
    for sub in nested:
        yield from sub     # delegate to another iterable
```

`itertools` — the standard toolbox: **[skim]**

```
from itertools import (count, cycle, repeat, chain, islice, product,
                       permutations, combinations, groupby, accumulate,
                       pairwise, takewhile, dropwhile)
chain([1,2], [3])            # 1 2 3
islice(count(), 5)           # first 5 of an infinite counter
product([1,2], repeat=2)     # (1,1) (1,2) (2,1) (2,2)
combinations([1,2,3], 2)     # (1,2) (1,3) (2,3)
accumulate([1,2,3])          # 1 3 6  — running totals
pairwise([1,2,3])            # (1,2) (2,3)      3.10+
groupby(sorted(xs, key=f), key=f)   # requires pre-sorting!
```

---

9. Functions

```
def f(a, b=10, *args, **kwargs):
    """Docstring."""
    return a + b                    # bare `return` / falling off end -> None

def g(): return 1, 2                # returns a tuple
x, y = g()

f(1, 2)                             # positional
f(a=1, b=2)                         # keyword
f(*[1, 2]); f(**{"a": 1, "b": 2})   # unpack into arguments

def h(a, /, b, *, c):               # a: positional-only; c: keyword-only
    ...
```

GOTCHA — mutable default arguments are evaluated ONCE at def time:

```
def bad(x, acc=[]):        # NEVER
    acc.append(x); return acc
bad(1); bad(2)             # [1, 2] — same list

def good(x, acc=None):
    acc = [] if acc is None else acc
```

Lambdas — single expression, no statements:

```
key = lambda r: (r.arrival, r.id)
sorted(xs, key=lambda r: -r.size)
```

Closures and decorators: **[skim for the OA, but recognize them]**

```
def make_counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc

import functools

def logged(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        print(f"calling {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper

@logged                    # equivalent to: f = logged(f)
def f(): ...

@functools.lru_cache(maxsize=None)     # memoization, free
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
```

`functools.reduce`, `partial`, and `cmp_to_key` round out the module.

---

10. Sorting — the most test-relevant topic [OA]

```
xs.sort()                                   # in place, returns None
ys = sorted(xs)                             # new list
sorted(xs, reverse=True)

sorted(xs, key=lambda r: r.arrival)                 # single key
sorted(xs, key=lambda r: (r.arrival, r.id))         # multi-key
sorted(xs, key=lambda r: (-r.priority, r.arrival, r.id))   # mixed direction
sorted(xs, key=len)                                 # any callable
sorted(d.items(), key=lambda kv: (-kv[1], kv[0]))   # by value desc, key asc
```

Rules to internalize:

1. A tuple key compares left to right — first field decides, later fields break ties.
2. Negate a numeric field to make it descending. `reverse=True` flips **every** field, which is usually not what a spec asks for.
3. For descending on non-numerics, sort twice (stable sort preserves the earlier order) or use `functools.cmp_to_key`.
4. `sorted` is **stable**: equal keys keep input order. Convenient, but when the spec states a tie-break, write it into the key explicitly.

Selection with the same discipline:

```
min(xs); max(xs)
min(xs, key=lambda r: (r.load, r.index))
```

```
min(range(len(loads)), key=lambda i: (loads[i], i))   # ties -> lowest index
max(items, key=lambda r: (-r.priority, r.id))         # ties -> highest id
```

Binary search on a sorted list:

```
import bisect
bisect.bisect_left(xs, v)      # leftmost insertion point
bisect.bisect_right(xs, v)     # rightmost
bisect.insort(xs, v)           # insert keeping order  (O(n) move)
```

---

11. collections, heapq, and friends [OA]

```
from collections import deque, defaultdict, Counter, OrderedDict, ChainMap

q = deque([1, 2, 3])
q.append(4); q.appendleft(0)        # O(1) both ends
q.pop(); q.popleft()                # O(1) both ends
q[0]; len(q); q.rotate(1); deque(maxlen=5)

d = defaultdict(list);  d[k].append(v)      # missing key -> []
d = defaultdict(int);   d[k] += 1           # missing key -> 0
d = defaultdict(set);   d[k].add(v)

c = Counter("aabbbc")               # {'b':3,'a':2,'c':1}
c.most_common(2); c["z"]            # 0 for missing, no KeyError
c.update("ab"); c1 + c2; c1 - c2; c.total()
```

```
import heapq
h = []
heapq.heappush(h, (priority, tiebreak_id, payload))
priority, _, payload = heapq.heappop(h)      # SMALLEST first
h[0]                                         # peek
heapq.heapify(xs)                            # O(n), in place
heapq.heappushpop(h, item); heapq.heapreplace(h, item)
heapq.nsmallest(3, xs, key=...); heapq.nlargest(3, xs, key=...)
```

Max-heap trick: push `-value`, or push `(-priority, id, obj)`. **Always include a deterministic tiebreak field before any non-comparable payload**, or Python will try to compare the payloads and crash.

---

12. Files, JSON, and I/O [skim for the OA]

```
with open("f.txt") as fh:            # context manager closes it for you
    text = fh.read()
    # fh.readline(); fh.readlines(); for line in fh: ...

with open("f.txt", "w") as fh:       # 'r' 'w' 'a' 'x', add 'b' for binary
    fh.write("hi\n")
    fh.writelines(["a\n", "b\n"])

import json
json.dumps(obj, indent=2, sort_keys=True); json.loads(s)
json.dump(obj, fh); json.load(fh)
```

```
import csv
reader = csv.DictReader(fh); writer = csv.DictWriter(fh, fieldnames=[...])

from pathlib import Path
p = Path("dir") / "f.txt"
p.exists(); p.read_text(); p.write_text("x"); p.stem; p.suffix; p.parent
list(Path(".").glob("**/*.py"))

import sys
sys.argv; sys.exit(1); print("err", file=sys.stderr)
input("prompt: ")
```

---

13. Exceptions [OA]

```
try:
    risky()
except (KeyError, IndexError) as e:
    print(type(e).__name__, e)
except Exception as e:              # never bare `except:`
    raise                           # re-raise, preserving traceback
else:
    print("no exception occurred")
finally:
    cleanup()                       # always runs

raise ValueError("message")
raise ValueError("msg") from original_error
assert cond, "message"              # AssertionError; stripped under -O

class MyError(Exception):
    pass
```

Common built-ins: `ValueError`, `TypeError`, `KeyError`, `IndexError`, `AttributeError`, `ZeroDivisionError`, `StopIteration`, `NotImplementedError` (the stub marker you'll be replacing in the OA).

EAFP is idiomatic Python — try it and catch the failure, rather than checking first:

```
try:                        # EAFP: Easier to Ask Forgiveness than Permission
    v = d[k]
except KeyError:
    v = default
v = d.get(k, default)       # ...though here the direct method is better still
```

---

14. Classes — full syntax [OA] (detail in the Day 2 refresher)

```
class Worker:
    """Docstring."""
    registry = []                   # CLASS attribute — shared by all!

    def __init__(self, index):
        self.index = index          # instance attribute
        self.running = []           # mutable state belongs here
```

```
    def load(self):                  # instance method
        return len(self.running)

    @property
    def is_full(self):               # accessed WITHOUT parens
        return self.load() >= 4

    @is_full.setter
    def is_full(self, value): ...

    @staticmethod
    def helper(x):                   # no self — just namespaced
        return x * 2

    @classmethod
    def from_config(cls, cfg):       # alternate constructor
        return cls(cfg.index)

    def __repr__(self):  return f"Worker({self.index})"
    def __str__(self):   return f"W{self.index}"
    def __eq__(self, o): return self.index == o.index
    def __hash__(self):  return hash(self.index)
    def __lt__(self, o): return self.index < o.index
    def __len__(self):   return len(self.running)
    def __iter__(self):  return iter(self.running)
    def __contains__(self, x): return x in self.running
    def __getitem__(self, i):  return self.running[i]
    def __bool__(self):  return bool(self.running)
    def __call__(self, x): ...
    def __enter__(self); def __exit__(self, *exc)     # context manager
```

Inheritance:

```
class Base:
    def pick(self): raise NotImplementedError

class Fifo(Base):
    def __init__(self, cfg):
        super().__init__()
        self.cfg = cfg
    def pick(self): return self.queue[0]

isinstance(x, Base); issubclass(Fifo, Base)

from abc import ABC, abstractmethod
class Scheduler(ABC):
    @abstractmethod
    def pick(self): ...              # cannot instantiate without overriding
```

Dataclasses:

```
from dataclasses import dataclass, field, asdict, replace

@dataclass(frozen=False, order=False, slots=True)
class Request:
    id: int
    arrival: int
    tokens: int = 0
    log: list = field(default_factory=list)     # mutable default REQUIRES this
```

```
    _cache: dict = field(default_factory=dict, repr=False, compare=False)

    @property
    def reserved(self): return self.tokens * 2

    def __post_init__(self):         # runs after generated __init__
        assert self.tokens >= 0

asdict(r); replace(r, tokens=5)      # convert / copy-with-changes
```

@dataclass generates __init__, __repr__, __eq__. Because eq=True it sets __hash__ = None → **instances are unhashable**; use frozen=True if you need them in a set. order=True generates < etc.

Enums: **[skim]**

```
from enum import Enum, auto
class State(Enum):
    WAITING = auto()
    RUNNING = auto()
State.WAITING.name; State.WAITING.value; list(State)
```

---

15. Modules, imports, and typing

```
import math
import numpy as np
from collections import deque
from collections import deque as dq
from mypkg.mod import thing
from . import sibling          # relative import inside a package

if __name__ == "__main__":     # runs only when executed directly
    main()
```

```
from typing import Optional, Any, Callable, Iterable, Iterator, Union

def f(x: int, ys: list[str], d: dict[str, int]) -> Optional[int]: ...
def g(cb: Callable[[int], str], xs: Iterable[int]) -> Iterator[int]: ...
x: int | None = None           # 3.10+ union syntax
```

Type hints are never enforced at runtime — they're documentation plus tooling.

---

16. Copying, identity, and mutability [OA]

The concept that causes the most confusion coming from other languages: **names are references, assignment never copies.**

```
a = [1, 2]; b = a; b.append(3);  a        # [1, 2, 3] — same object
a is b                                     # True

import copy
b = a[:]  or  a.copy()  or  list(a)        # shallow — nested objects shared
b = copy.deepcopy(a)                       # fully independent
```

```
grid = [[0] * 3] * 3                        # three references to ONE row
grid[0][0] = 1; grid                        # [[1,0,0],[1,0,0],[1,0,0]]
```

Function arguments are passed by reference-to-object: mutating a list argument affects the caller; rebinding the name inside the function does not.

```
def f(xs):
    xs.append(1)      # visible to the caller
    xs = [9]          # NOT visible — rebinds the local name only
```

Immutable: `int, float, str, bytes, tuple, frozenset, bool, None`. Mutable: `list, dict, set, bytearray`, and most class instances.

---

17. Iteration protocol & useful built-ins

```
iter(xs); next(it); next(it, default)

class Countdown:
    def __init__(self, n): self.n = n
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0: raise StopIteration
        self.n -= 1
        return self.n + 1
```

```
len, sum, min, max, abs, round, sorted, reversed, enumerate, zip
any, all, map, filter, range, type, isinstance, id, hash, repr
int, float, str, bool, list, tuple, dict, set, frozenset
divmod, pow, format, print, input, open, dir, vars, getattr, setattr
```

`map/filter` are rarely idiomatic — prefer comprehensions:

```
list(map(str, xs))        ==  [str(x) for x in xs]
list(filter(None, xs))    ==  [x for x in xs if x]
```

---

18. Debugging techniques for a timed test [OA]

```
print(f"{t=} {reserved=} running={[r.id for r in running]}")   # f-string debug
assert reserved >= 0, f"negative memory at t={t}"

import pprint; pprint.pprint(complex_structure)
print(vars(obj))          # instance __dict__ — all attributes at once
print(dir(obj))           # every available member — great for unfamiliar APIs
help(obj.method)          # docstring, offline

import traceback; traceback.print_exc()
breakpoint()              # drops into pdb: n(ext) s(tep) c(ont) p(rint) q(uit)
```

In a tick-based simulation, printing one line per tick showing the clock, queue, running set, and memory is the fastest possible way to find an off-by-one. Write it early, delete it before submitting.

---

19. Complexity cheat sheet [OA]

Operation	list	deque	dict/set	heapq
index / key lookup	O(1)	O(1) ends	O(1)	—
append / push	O(1)*	O(1)	O(1)	O(log n)
pop from end	O(1)	O(1)	—	O(log n)
pop from front	**O(n)**	O(1)	—	—
insert / delete middle	O(n)	O(n)	O(1) by key	—
`in` membership	**O(n)**	O(n)	O(1)	—
min element	O(n)	O(n)	O(n)	**O(1)**
sort	O(n log n)	—	—	—

* amortized. The two decisions that matter in practice: use `deque` when you pop from the front, and use a `set`/`dict` when you test membership in a loop.

---

20. Twelve traps, collected [OA]

1. `a = b` doesn't copy; `b.append()` mutates both.
2. `[[0]*3]*3` aliases rows — use a comprehension.
3. Mutable default arguments persist across calls.
4. `list: list = []` in a dataclass raises — use `field(default_factory=list)`.
5. Mutating a list while iterating it silently skips elements.
6. `-7 // 2 == -4`, and `int(a/b)` ≠ `a // b` for negatives.
7. `0.1 + 0.2 != 0.3` — use `math.isclose`.
8. `xs.sort()` returns `None`; `sorted(xs)` returns the list.
9. `reverse=True` flips all sort keys, not just one — negate instead.
10. `@property` members are accessed without parentheses; calling one raises a confusing `TypeError` far from the real mistake.
11. `@dataclass` instances are unhashable unless `frozen=True`.
12. Heap pushes of bare objects crash on ties — push tuples with an id.

Practice

`day1_drills.py` — 10 exercises covering the sections marked **[OA]**. Run it, fill in the TODOs, check against `day1_drills_answers.py`.